

MDMC
Master in Data Management and Curation

AREA SCIENCE PARK

SISSA

Python for Data Management I, 2026

Valerio Piomponi, Ahmed Khalil, Elaheh Saadat





Py4DM-I - Program

26

27

Exercise: Metadata extraction from a SEM image

Pythonk Packages
Collaboration with **Git**

28

29

Introduction to Web Development **(Django)**

6

Exercise: Implementing REST APIs In DPC explorer

Django REST APIs

JSON (JavaScript Object Notation)

- Plain-text format for representing **structured data**
- From JavaScript Synthax to write Objects, now **universal**
- **Objects**: key–value pairs in curly braces
  Arrays: ordered lists in square brackets
- **Nested** to any Depth
- Standard format for storing **Metadata**
- Human-readable and machine-readable at the same time
- **Native language of the web**: Using JSON means data flows from the database to the API to the browser with essentially no translation


```
{
  "filename": "scan_0042.tiff",
  "format": "TIFF",
  "width": 4960,
  "height": 7016,
  "color_space": "CMYK",
  "compression": "LZW",
  "pages": 3,
  "created": "2026-10-26T09:15:00Z",
  "tags": ["invoice", "archive", "2026"],
  "source": {
    "device": "Epson DS-870",
    "dpi": 600
  }
}
```



```
{
  "filename": "scan_0042.tiff",
  "source": {
    "device": {
      "model": "Epson DS-870",
      "settings": {
        "dpi": 600,
        "color": {
          "mode": "CMYK",
          "profiles": ["ISO Coated v2", "Fogra39"]
        }
      }
    }
  }
}
```

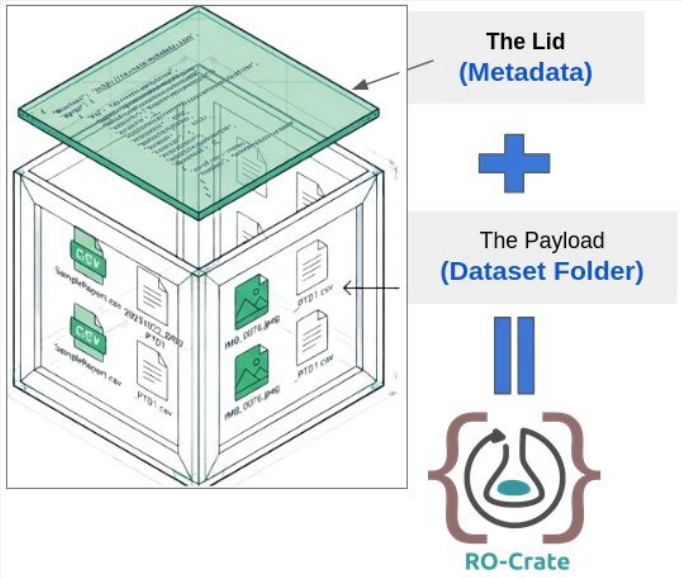
RO-Crate (Research Object Crate)

Framework to package research data together with its metadata.

Ro-crate folder: A folder of research data plus one JSON-LD file that describes it in a standard, machine-readable way.

JSON-LD file(`ro-crate-metadata.json`): describes the dataset, its files, and its context (who made it, how, with what tools, and under which licence),

use schema.org terms (shared vocabulary for the web



Why publish a dataset with RO-Crate?

Findable & citable

Title, authors, licence and publisher in schema.org terms that repositories and search engines can index.

Self-contained

Metadata travels with the data in one folder or zip, with an optional human-readable ro-crate-preview.html.

Machine-actionable

JSON-LD gives every statement a defined meaning, so tools can validate, harvest and merge crates automatically.

Rich provenance

Records who created each file, with which software, instruments and workflows, not just the final numbers.

Clear reuse terms

Licences attach to the whole crate or to single files, so reusers know exactly what they may do.

Lightweight & supported

Plain JSON, no new infrastructure. Libraries such as ro-crate-py and platforms such as WorkflowHub already use it.

CROSS-REFERENCE

ro-crate-metadata.json · excerpt

```
{ "@context": "https://w3id.org/ro/crate/1.2/context",
  "@graph": [
  { "@id": "./", "@type": "Dataset",
    "hasPart": [{ "@id": "data.csv" }],
    "author": { "@id": "https://orcid.org/0000-0002-1825-0097" },
    "license": { "@id": "http://spdx.org/licenses/CC-BY-4.0" } },
  { "@id": "data.csv", "@type": "File",
    "author": { "@id": "https://orcid.org/0000-0002-1825-0097" },
    "encodingFormat": "text/csv" },
  { "@id": "https://orcid.org/0000-0002-1825-0097",
    "@type": "Person", "name": "Josiah Carberry",
    "affiliation": { "@id": "https://ror.org/05gq02987" } },
  { "@id": "http://spdx.org/licenses/CC-BY-4.0",
    "@type": "CreativeWork", "name": "CC BY 4.0" },
  { "@id": "https://ror.org/05gq02987",
    "@type": "Organization", "name": "Brown University" }
  ] }
```

1 · Identify

Each entity in the @graph has a unique @id: a relative path for files, a web IRI (ORCID, ROR, SPDX) for people, organisations and licences.

2 · Reference

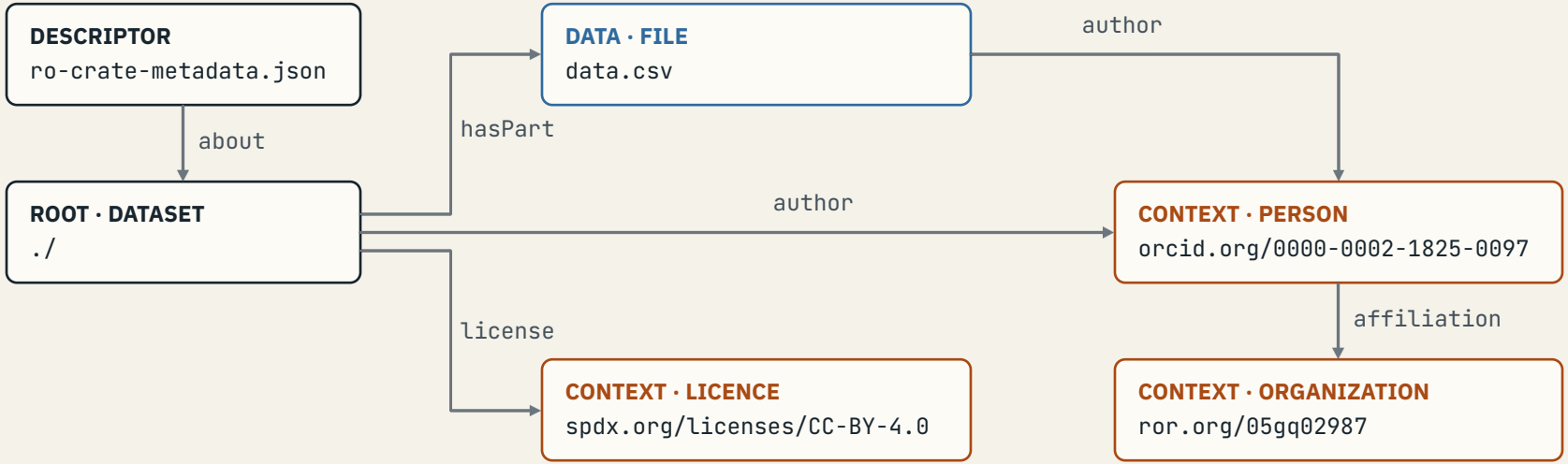
A value written as { "@id": … } is a link: it points to another entity instead of repeating its details.

3 · Reuse

Describe once, link many times: the dataset and data.csv share one Person entity.

- Blue: defines an entity
- Orange, bold: references an entity

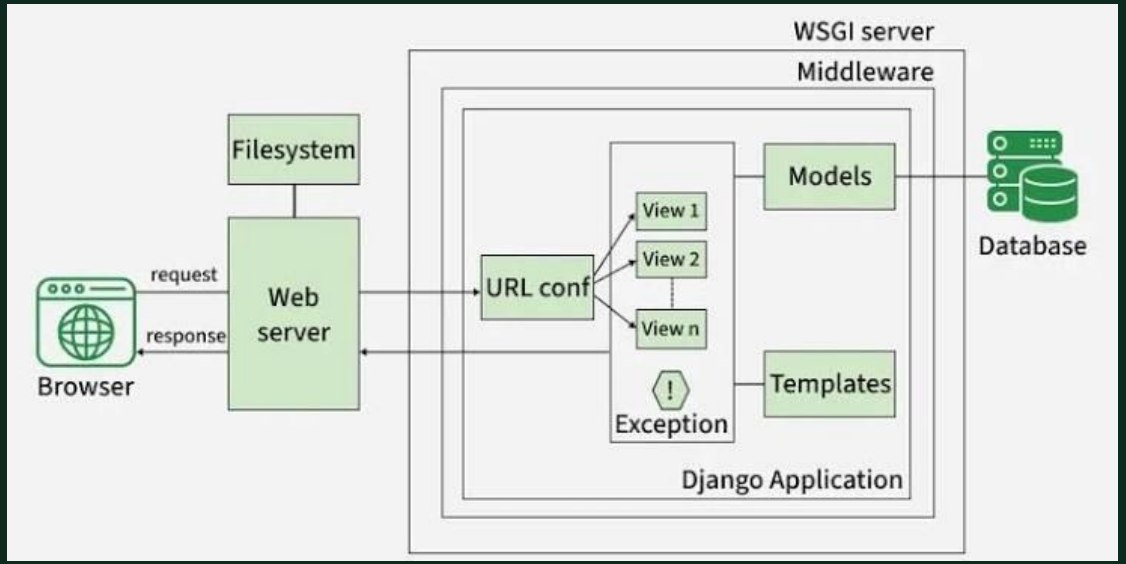
CROSS-REFERENCE



Follow the @id links to answer “who made data.csv, and where do they work?” Each entity is described once in the flat @graph; the links connect them.

What is Django?

Open Source Web framework written in Python





The client–server model

Client
Browser, mobile app or Python script

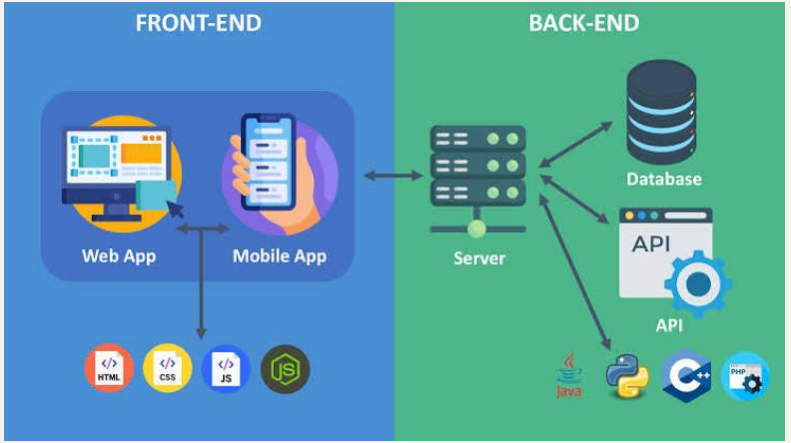
HTTP request →
← HTTP response

Server
Django application

SQL query →
← rows

Database
SQLite, PostgreSQL…

The client always speaks first: a server only ever answers requests.



What a web framework does for you

Routing
Send each URL to the Python code that handles it

Data
Store and query data without writing raw SQL

Security
Protection against common attacks by default

Users
Login, sessions, permissions and passwords

Output
Turn data into HTML pages or JSON

Tooling
Dev server, admin site, tests, migrations

Why Django?

“The web framework for perfectionists with deadlines.”

“Batteries Included”

- **Shared maintenance:** let’s have a unique common framework in our lab!
- **Easier onboarding**
- **Reusable code.** Apps written for one service, such as authentication, a common data model can be moved into another.
- **Simpler infrastructure.** Deployment, updates and security patches follow one routine instead of several
- **Shared expertise.** When something breaks, there’s a person down the corridor who has seen the problem before.

Anatomy of a URL

(Uniform Resource Locator) address of something on the internet

https://rest.uniprot.org/uniprotkb/P69905?format=json

Scheme

`https://`

The protocol: HTTP over TLS

Host

`rest.uniprot.org`

Which server to talk to

Domain name, Behind the scenes gets translated into IP address through the Domain Name System (DNS)

Path

`/uniprotkb/P69905`

Which resource on that server

Query

`?format=json`

Extra parameters, key=value

An HTTP request and its response

HyperText Transfer Protocol

```
Request: line · headers · body
GET /rest/v1/core/entry/4HHB HTTP/1.1
Host: data.rcsb.org
Accept: application/json
User-Agent: python-requests

(no body for a GET)
```

Structured text indicating:
Method (GET), path, headers with extra details

```
Response: status · headers · body
HTTP/1.1 200 OK
Content-Type: application/json

{
  "rcsb_id": "4HHB",
  "struct": {
    "title": "THE CRYSTAL STRUCTURE OF HUMAN…"
  },
  "exptl": [{"method": "X-RAY DIFFRACTION"}],
  …
}
```

Status code, headers, and the the body

HTTP methods say what you want to do

Method	Purpose	Example
GET	Read a resource or a list	GET /api/searches/
POST	Create a new resource	POST /api/searches/
PUT	Replace a resource entirely	PUT /api/searches/42/
PATCH	Update part of a resource	PATCH /api/searches/42/
DELETE	Remove a resource	DELETE /api/searches/42/

Plain HTTP sends everything as readable text

HTTPS sent through an encrypted layer called **TLS**

Status codes say what happened

2xx
Success
- 200 OK
- 201 Created
- 204 No Content

3xx
Redirection
- 302 Found
- 304 Not Modified

4xx
Client error
- 400 Bad Request
- 401 Unauthorized
- 404 Not Found

5xx
Server error
- 500 Internal Error
- 502 Bad Gateway
- 503 Unavailable



HTTP is the protocol; HTML and JSON are payloads

how data travels, and *what* the data actually is

HTML (HyperText Markup Language)

is for humans: a browser renders it as a page.



```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Title goes here</title>
  </head>
  <body>

  </body>
</html>
```

HTTP is the protocol; HTML and JSON are payloads

how data travels, and *what* the data actually is

```
Content-Type: text/html
<h1>Haemoglobin</h1>
<p>PDB ID: <b>4HHB</b></p>
<p>Method: X-ray diffraction</p>
```

HTML (HyperText Markup Language)

is for humans: a browser renders it as a page.

```
Content-Type: application/json
{
  "pdb_id": "4HHB",
  "name": "Haemoglobin",
  "method": "X-RAY DIFFRACTION"
}
```

JSON is for programs: code parses it into data.

Frontend and backend

Frontend

Runs in the user's browser

- HTML, CSS, JavaScript
- Frameworks: React, Vue, Svelte
- Displays data, handles clicks



Backend

Runs on a server

- Python + Django
- Business logic, validation
- Database, users, external APIs

Create the project

https://docs.djangoproject.com/en/6.1/intro/tutorial01/

Terminal

```
$ django-admin startproject mysite djangotutorial
```

What it creates

```
djangotutorial/
  manage.py
  mysite/
    __init__.py
    settings.py
    urls.py
    asgi.py
    wsgi.py
```

mysite is the Python package holding your settings. You import from it, as in mysite.urls.

djangotutorial is just the outer folder. Django doesn't care what it's called.

Avoid names like **django** or **test**: they clash with Django and with Python's built-in test package.

What each project file does

File	Its job
manage.py	Command-line tool for this project: runserver, startapp, migrate...
mysite/__init__.py	Empty file that marks the folder as a Python package
mysite/settings.py	Configuration: installed apps, database, secret key, debug mode
mysite/urls.py	The root URL list: the site's "table of contents"
mysite/asgi.py, wsgi.py	Entry points for production web servers (ASGI or WSGI)

Keep an eye on **urls.py**: it is where every request starts its journey.

Manage.py

`os` set an environment variable
`sys` is used to read the command-line arguments you typed.

Django needs to know where your project's configuration lives

Import Django

```python
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys


def main():
    """Run administrative tasks."""
    os.environ.setdefault("DJANGO_SETTINGS_MODULE", "mysite.settings")
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)


if __name__ == "__main__":
    main()
```

only run `main()` when this file is executed directly.

Try django-admin runserver
Then try python manage.py runserver

Settings.py

```python
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent


# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = "django-insecure-qjihq4xf4ef=qp_n@4c8-ly5gvt(d(h!e@$+5hj$ecfn(dtf08"

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []


# Application definition

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
```

Projects and apps

Project

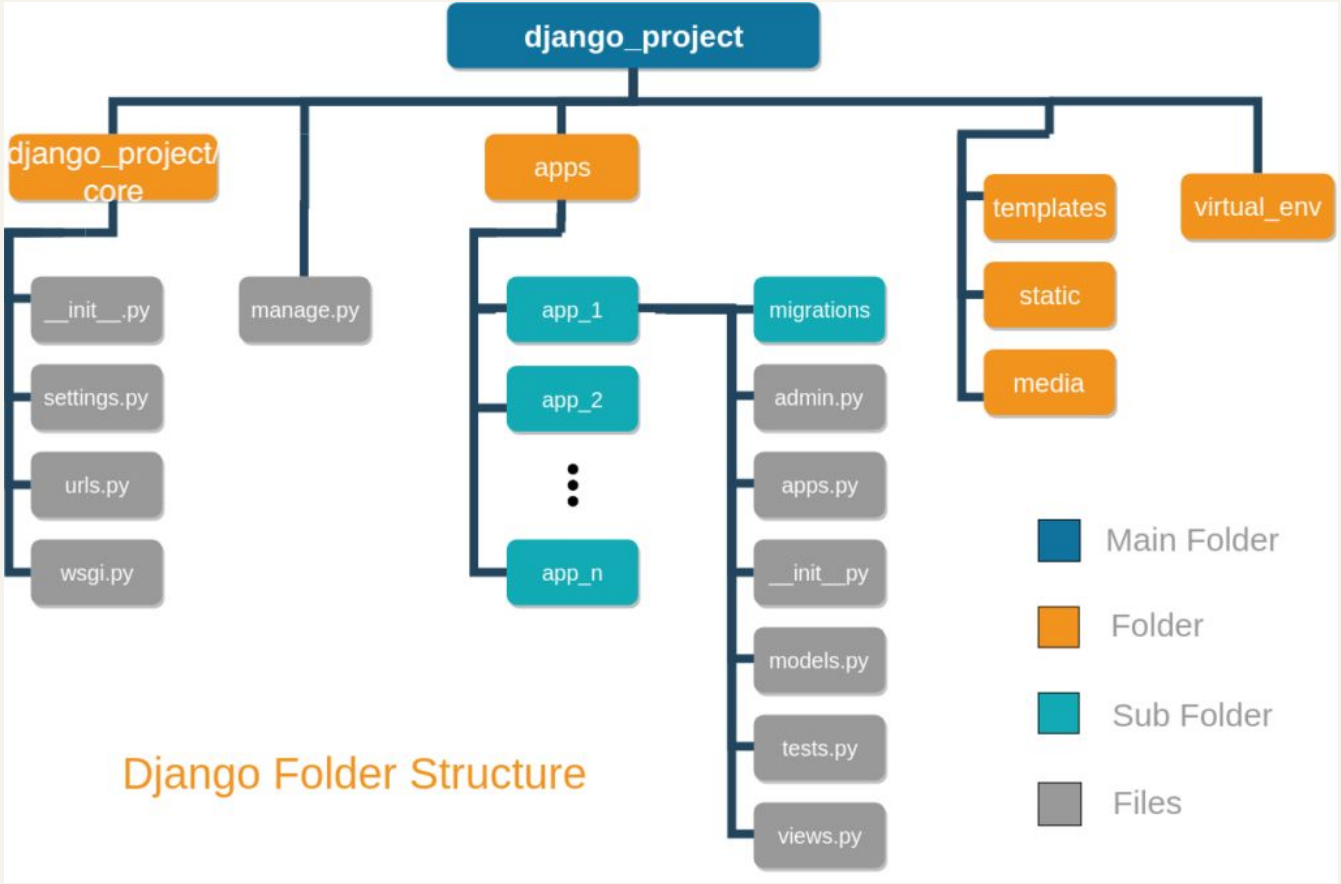
The whole site: settings, root URLs, deployment entry points.

App

One feature, self-contained and reusable: its own models, views and URLs.

One project contains many apps. Our exercise needs one app: **proteins**.

Projects and apps



Create the polls app

Next to manage.py

```
$ python manage.py \
  startapp polls

polls/
  __init__.py
  admin.py
  apps.py
  migrations/
  models.py
  tests.py
  views.py
```

Django generates the standard layout of an app, so you write code instead of creating folders.

File	Used for
views.py	What each page returns (today)
urls.py	Not generated: you'll create it
models.py	Your data tables (part 2)
admin.py	Admin site setup (part 2)
migrations/	Database change history

One idea behind every Django page



URLs decide which code runs: **address.**

Views decide what comes back: **answer.**

view.py

polls/views.py

```python
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello, world. You're at the polls index.")
```

Request in

Django passes the request: URL, method, headers.

Response out

A view must return an HttpResponse. (or a Subclass of it)

Class	Status	Use
`HttpResponse`	200	Plain text/HTML
`JsonResponse`	200	JSON data (APIs, AJAX)
`HttpResponseRedirect`	302	Temporary redirect
`HttpResponsePermanentRedirect`	301	Permanent redirect
`HttpResponseNotModified`	304	Caching
`HttpResponseBadRequest`	400	Invalid input
`HttpResponseForbidden`	403	No permission
`HttpResponseNotFound`	404	Missing resource
`HttpResponseNotAllowed`	405	Wrong HTTP method

Map a URL to the view

polls/urls.py · new file

```
from django.urls import path

from . import views

urlpatterns = [
    path("", views.index, name="index"),
]
```

route ""
The piece of URL to match. Empty means the app's own root.

view views.index
The function to call. No parentheses: you hand it over, Django calls it.

name "index"
A label for this URL, so other code can refer to it.

Plug the app into the project

mysite/urls.py · the root URLconf

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("polls/", include("polls.urls")),
    path("admin/", admin.site.urls),
]
```

include() delegates

Django chops off the part that matched, **polls/**, and passes what's left to polls/urls.py.

The rule

Always use include() for another app's URLs. The one exception is admin.site.urls.

From dajngo.shortcuts import render

`render()` merges that data into a **template** to produce the HTML page.

Browser → URL → View → render() → Template → HTML → Browser

Template: text file (HTML)
defines the **structure and layout** of a page

```python
from django.shortcuts import render
from django.http import HttpResponse

def index(request):
    #return HttpResponse("Hello, world. You're at the polls index.")
    return render(request, "index.html", {'name': 'index'})

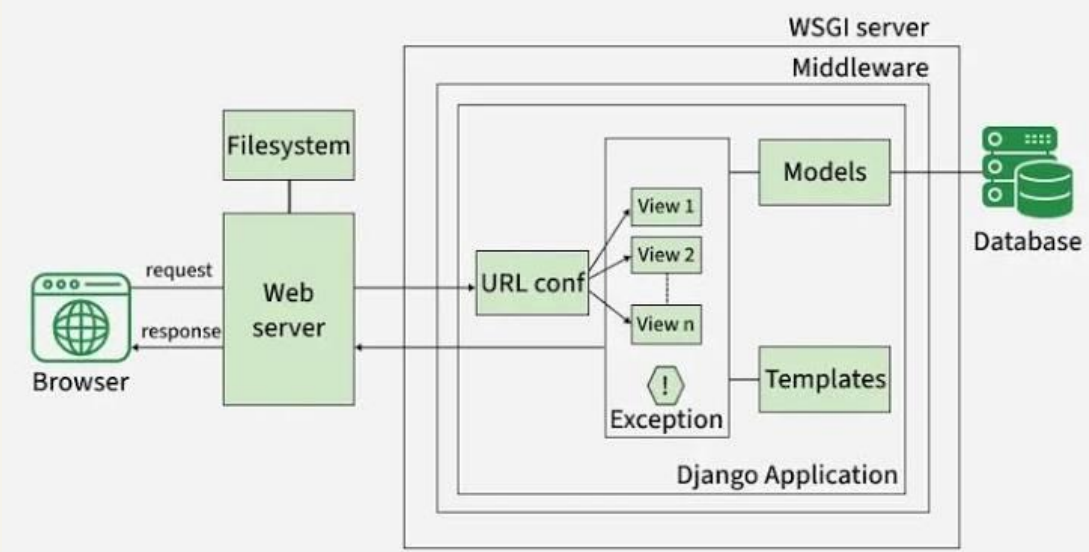
def detail(request):
    #return HttpResponse("Hello, world. You're at the polls index.")
    return render(request, "index.html", {'name': 'detail'})
# Create your views here.
```

polls > templates > index.html > h1

```
1   <h1>Hello, world. You're at the polls {{ name }}</h1>
```

Models.py

- Single source of truth for your data.
- Structure defined once in Python
- Django creates the tables, provides the query API, and builds the admin interface from it.




```
                ┌──> Database tables (via migrations)
models.py ──> Shell and views (via Book.objects)
                └──> Admin forms (via admin.site.register)
```

Models.py

```python
from django.db import models

# Create your models here.
class Question(models.Model):
    question_text = models.CharField(max_length=200)
    pub_date = models.DateTimeField("date published")

class Choice(models.Model):
    question = models.ForeignKey(Question, on_delete=models.CASCADE)
    choice_text = models.CharField(max_length=200)
    votes = models.IntegerField(default=0)
```

Class → table

Question becomes a database table.

Field → column

Each attribute has a type and options.

Instance → row

One object is one record.

Migrations keep the database in sync

1	`edit models.py`	Add or change a model or field
2	`python manage.py makemigrations`	Django writes a migration file
3	`App/migrations/0001_initial.py`	A versioned recipe of the change
4	`python manage.py migrate`	The database tables are updated

Migration files are code: commit them with the rest of your project.

Why 2 separates command? Think Git

one person *write down* a database change, and then everyone else (and every server) *applies* that same change